

SIMATSENGINEERING

ASSESSMENTTOOL3 COURSE

NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CPU utilization
and

CO2: Analyze process management and deadlock handling techniques to improve system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

Corrected Code:

Process P1:

```
lock(R1)

wait(100 ms)

lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R1)

wait(100 ms)

lock(R2)
```

critical section

unlock(R2)

unlock(R1)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions: *ready* →

waiting → *ready* → *waiting* → *ready* ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

Corrected Scheduler Log:

Ready → Running → I/O Waiting



Another Ready Process Executes



I/O Completion → Ready → Running

Changes Made

- Instead of leaving the CPU idle during I/O waiting, schedule another ready process.
- Avoid continuous switching between Ready and Waiting.

Optimization

- Use Asynchronous I/O.
- Use Multilevel Feedback Queue (MLFQ) scheduling.

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

Corrected Memory Map:

Code = 5 MB

Heap = Optimized (Reduce Unused Memory)

Stack = 512 KB per Thread

Threads = 15

RAM = 2 GB

Reduced Page Faults

Changes Made

- Reduced unnecessary heap allocation.
- Reduced number of active threads.
- Reduced stack size per thread.

Optimization

- Optimize heap memory.
- Reduce active threads.

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

Corrected Queue:

Q1 → Interactive

Q2 → System

Q3 → Batch (P5)

If P5 waits too long



Increase Priority



Move P5 to Q2



CPU Allocated

Changes Made

- Apply **Aging**.
- Increase priority of waiting processes.

Optimization

- Use Aging.
- Dynamically increase priority

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

Corrected:

Burst Time = 2 ms

Time Quantum = 4 ms

Reduced Context Switches

Improved CPU Performance

Changes Made

- Increased time quantum.
- Reduced unnecessary preemption.

Optimization

- Increase time quantum.
- Reduce unnecessary thread switching.

Q6. Semaphore Synchronization Debugging

A producer-consumer application occasionally hangs during execution. The semaphore values observed are: mutex = 1 empty = 0 full = 10 Buffer size = 10

Producer Code:

wait(mutex)

wait(empty)

produce item

signal(full)
signal(mutex)

Consumer Code:

wait(mutex)
wait(full)
consume item
signal(empty)
signal(mutex)

Identify the synchronization bug causing the system to hang.

Explain which deadlock condition is being violated.

Suggest an optimization to ensure correct synchronization without changing the producer consumer functionality.

Corrected Producer:

wait(empty)
wait(mutex)
produce item
signal(mutex)
signal(full)

Corrected Consumer

wait(full)
wait(mutex)
consume item
signal(mutex)
signal(empty)

Changes Made

- Semaphore checked before mutex.
- Mutex released before signaling.
- Prevents deadlock.

Deadlock Condition Violated

- **Hold and Wait**

Optimization

- Acquire semaphores in the correct order.
- Release mutex immediately after critical section.

Q7. Thrashing Detection and Optimization

A virtual memory system reports the following statistics:

Number of processes: 50

Average page fault rate: 250 faults/sec

CPU Utilization: 18%

Disk Utilization: 95%

Free Frames Available: 2%

The operating system suspects thrashing.

Debug the root cause of the performance degradation.

Explain the relationship between page faults and CPU utilization.

Propose two operating system optimizations to reduce thrashing.

Corrected:

Processes = 30

Page Faults = 40/sec

CPU = 80%

Disk = 50%

Free Frames = 20%

Changes Made

- Reduced multiprogramming level.
- Increased available frames.

Optimization

- Working Set Model.
- Page Fault Frequency (PFF) control.

Q8. File System Performance Debugging

A database server experiences slow file access times. System logs indicate:

Average file size: 500 MB

Disk seeks per request: 120

Disk utilization: 92%

Cache hit ratio: 15%

Identify the file system bottleneck affecting performance.

Explain why disk access latency is increasing.

Suggest two optimizations to improve file access efficiency.

Corrected:

Disk Seeks = 35

Disk Utilization = 60%

Cache Hit Ratio = 85%

Changes Made

- Increased cache efficiency.
- Reduced disk seek operations.

Optimization

- Increase buffer cache.
- Use sequential file allocation.

Q9. Thread Scheduling Debugging

A web server creates one thread per client request. During peak hours:

Active Threads: 5,000

CPU Utilization: 60%

Memory Usage: 92%

Average Response Time: 12 seconds

Scheduler logs show frequent thread creation and termination.

Identify the primary issue affecting server performance.

Explain how thread management contributes to the delay.

Recommend two optimizations to improve scalability and response time.

Corrected:

Thread Pool = 300

Memory = 60%

Response Time = 2 sec

Reuse Existing Threads

Changes Made

- Replaced one-thread-per-request model.
- Reused worker threads.

Optimization

- Thread Pool.
- Limit maximum active threads.

Q10. Disk Scheduling Optimization Problem

A disk scheduling log shows the following request queue:

98, 183, 37, 122, 14, 124, 65, 67

Current Head Position: 53

The system uses FCFS scheduling and reports:

Average seek time: Very High

Throughput: Low

User complaints regarding slow response Debug why FCFS is causing poor performance in this scenario.

Compare FCFS with one suitable disk scheduling algorithm for this workload.

Propose an optimization to improve seek time and overall throughput.

Corrected Using SSTF:

Head = 53

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Average Seek Time = Low

Throughput = High

Changes Made

- Replaced FCFS with SSTF scheduling.
- Head moves to the nearest request first.

Optimization

- Use **SSTF (Shortest Seek Time First)**.
- Reduces seek time and improves throughput.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation